

# Chapter 7

## Retrieval-Augmented Generation



Filippo Bianchini 

**Abstract** Retrieval-augmented generation (RAG) has emerged as a transformative approach to enhancing the capabilities of large language models (LLMs) by integrating real-time information retrieval with generative text synthesis. Traditional LLMs rely on pre-trained data, limiting their effectiveness when handling tasks requiring up-to-date or domain-specific information. RAG addresses this constraint by embedding a retrieval mechanism within the LLM framework, allowing the model to access external knowledge sources dynamically. This integration enables LLMs to generate contextually relevant and factually accurate responses, extending their utility within the realm of information systems (IS), where timely and precise information is critical. Starting from the history and evolution of retrieval-augmented generation, passing through different framework and methodologies, and moving to practical applications by different libraries, in this chapter, we offer a theoretical and practical overview of this innovative and powerful technique.

### 7.1 Introduction

The origins of retrieval-augmented generation (RAG) trace back to the early developments in question-answering systems [5] from the 1970s, when researchers in information retrieval began building systems capable of using natural language processing (NLP) to retrieve answers from text collections. Initially, these systems were confined to narrow domains, but they laid the groundwork for modern techniques by enabling machines to understand and process human questions. An example from the mid-1990s is *Ask Jeeves*<sup>1</sup> [17], which introduced a conversational,

---

<sup>1</sup> *Ask Jeeves* was a search engine where users could ask questions in natural language: <https://www.ask.com/>.

---

F. Bianchini (✉)  
Sapienza Università di Roma, Rome, Italy  
e-mail: [bianchini@diag.uniroma1.it](mailto:bianchini@diag.uniroma1.it)

user-friendly question-answering service with a virtual assistant mascot. This helped popularize the idea that users could ask natural language questions and receive relevant answers, an important step toward modern information retrieval systems [12].

A significant advancement in AI-driven information retrieval occurred in 2011 when IBM's Watson [9] demonstrated the potential of advanced AI by defeating human champions on the quiz show *Jeopardy!*.<sup>2</sup> Unlike earlier question-answering systems that relied solely on predefined databases, Watson leveraged NLP and sophisticated retrieval and ranking algorithms to analyze vast amounts of unstructured text in real time.

As AI research progressed, the limitations of purely generative models became increasingly evident, particularly their tendency to produce factually incorrect or unverifiable information. Despite steady advancements in NLP, the fundamental principles of text mining and information retrieval remained critical for ensuring factual consistency. The challenge, therefore, was to develop systems that could generate humanlike responses while grounding their output in verifiable, up-to-date sources. In this context, another important milestone for RAG is the development of the transformer architecture [29], a fundamental advancement that enabled language models to integrate additional knowledge through pre-trained models (PTMs) [3, 19]. This progression and the demand for more reliable AI-generated content led Patrick Lewis to introduce RAG in a 2020 paper [19], marking a key evolution in generative AI. His work proposed a novel framework that seamlessly combines a retrieval mechanism with a language model to enhance the handling of knowledge-intensive NLP tasks. Specifically, RAG integrates two key components: a *retriever*, which fetches relevant documents from an external knowledge base, and a *generator*, which synthesizes responses based on the retrieved information. In an interview, Lewis expressed regret over the unflattering acronym, acknowledging that they would have chosen a more appealing name had they anticipated the widespread impact of their work. RAG has since evolved into a recognized framework that encompasses a growing array of methods utilized in numerous academic papers and commercial applications. An overview of Lewis' approach is presented in Fig. 7.1.

This hybrid approach bridges the gap between traditional retrieval-based systems and modern generative models, improving both factual accuracy and contextual relevance and prepared the way for modern RAG technologies, as we know it today, which enhance LLMs by integrating real-time external data for greater accuracy and relevance. Figure 7.2 illustrates the rapid evolution and expansion of RAG techniques, from the release of GPT-3 in 2020, through the launch of ChatGPT, to the introduction of GPT-4.

These models have transformed numerous tasks such as summarizing text, answering questions, and creative writing. The reach of LLMs is now pervasive,

---

<sup>2</sup> A popular American quiz show where contestants respond to general knowledge clues with questions, aiming to accumulate the most money by the end of each round: <https://www.jeopardy.com/>.

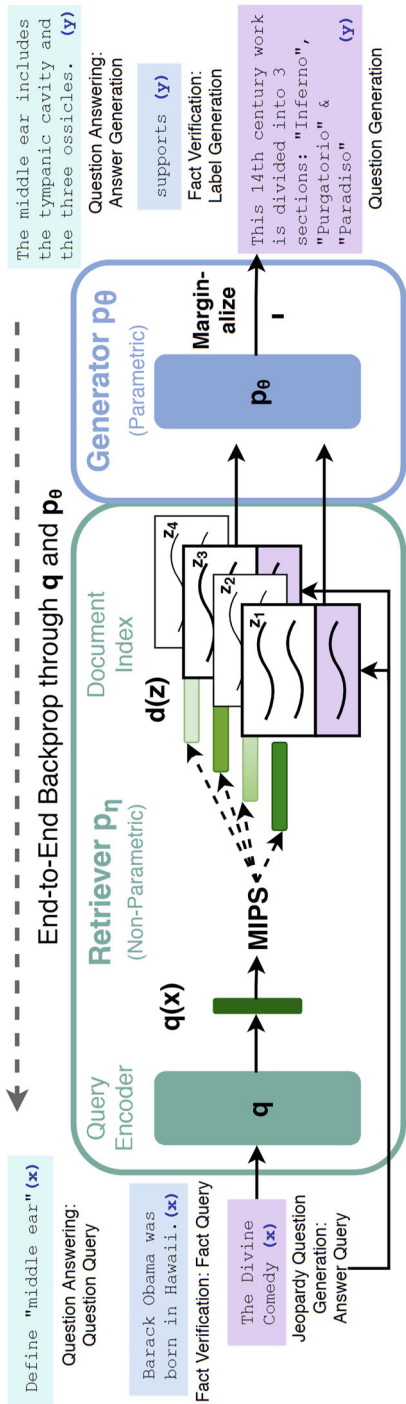
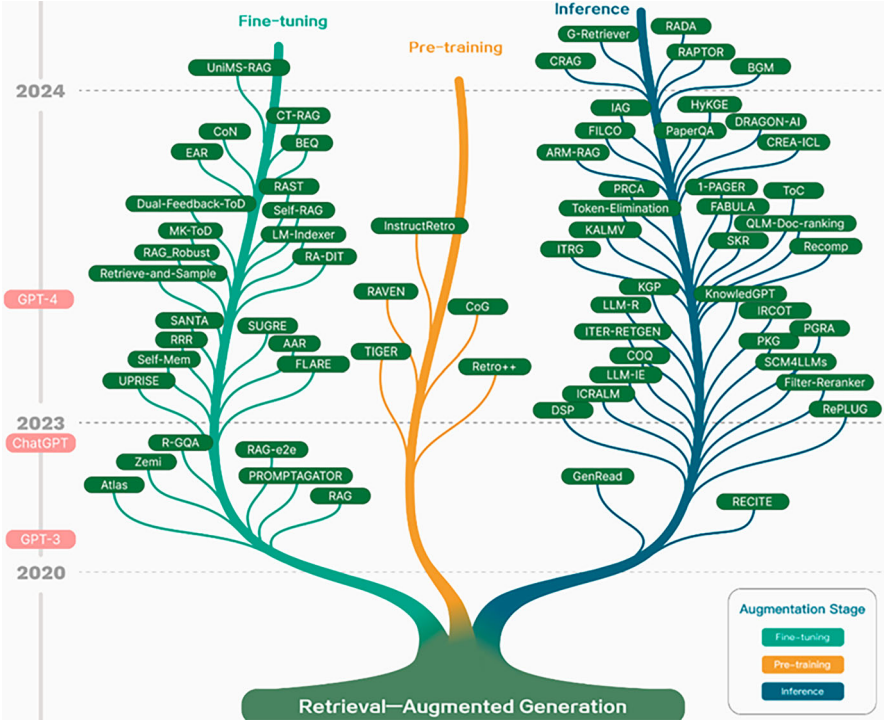


Fig. 7.1 Overview of Lewis' approach for RAG [19]



**Fig. 7.2** Technology tree of RAG research [10]

extending beyond expert users to a wider audience. As their adoption grows, so does our understanding of their strengths and limitations. A key issue that has surfaced is the tendency of LLMs to generate responses that, while coherent and convincing, may contain information that is partially or entirely incorrect. This phenomenon, often referred to as *hallucination* in the AI community, poses a significant challenge in high-stakes environments. Hallucination occurs due to the probabilistic nature of LLMs, which generate text based on probability rather than fact-checking [30]. As a result, LLMs sometimes *fill in* information in a way that sounds plausible but is factually wrong [16]. This design enables impressive fluency and flexibility but lacks an internal verification mechanism to cross-check factual accuracy.

When an LLM powered by RAG receives a prompt or question, it first consults an external information source, which could be a search engine, a database, or another structured repository. The retrieval component finds relevant documents, passages, or data points based on the prompt. Instead of generating an answer solely from its pre-trained language patterns, the LLM uses this retrieved information to guide and inform its response. The result is a hybrid output: the natural, conversational style of an LLM response combined with real-time data, significantly improving factual accuracy. With these advancements, RAG research shifted toward enhancing LLMs’ ability to handle complex, information-rich queries during the inference stage, leading to accelerated progress in RAG methodologies.

## 7.2 RAG: Working Logic

The typical RAG framework combines a retrieval mechanism with generative language models to enhance the quality and accuracy of responses in natural language processing tasks. The functioning involves a two-step process: *retrieving* and *generation*. Before these steps are executed, a preliminary phase takes place. This phase is essential for preparing and organizing the data, ensuring effective querying by the retrieval system.

### 7.2.1 Preliminary Step

The core workflow of RAG begins with several preparatory steps, including data loading, parsing, embedding, and indexing. These steps ensure that external knowledge sources can be efficiently queried by the retrieval mechanism.

#### 7.2.1.1 Data Loading and Parsing

The initial step in the RAG system involves loading the necessary data, which may originate from various sources and exist in different formats, such as PDF, HTML, Word, Markdown, or database exports. These various formats require processing to ensure compatibility with the retrieval system. Once loaded, the raw text undergoes a critical parsing phase, essential for achieving uniformity and ensuring clean, structured formatting suitable for indexing and querying. Parsing encompasses several key operations:

- Removing unnecessary metadata and encoding artifacts that might interfere with efficient processing
- Tokenizing the text into smaller, manageable segments to facilitate easier indexing and retrieval
- Splitting large documents into smaller, contextually meaningful chunks that preserve semantic coherence while optimizing retrieval performance

The goal of parsing is to prepare the raw data for transformation into a suitable embedding space and to ensure efficient indexing for similarity searches.

#### 7.2.1.2 Embedding

The next stage in the RAG system is converting the cleaned and preprocessed text into numerical representations using embedding models. Embeddings are vector representations of the text chunks and play a central role during the retrieval phase by enabling the system to compute semantic similarity between user queries and

indexed data [4]. Essentially, embeddings map text into a high-dimensional vector space, capturing their contextual and semantic meaning in a way that facilitates efficient comparison and matching.

Common embedding methods include a variety of pre-trained models that have proven effective in encoding semantic relationships. Among the most widely used methods are:

- **Word2Vec:** a model introduced by Mikolov et al. [21] that represents words as dense vectors<sup>3</sup> by analyzing their co-occurrence in large corpora. Word2Vec embeddings capture word similarity based on their contextual usage.
- **GloVe:** Global Vectors for Word Representation was developed by Pennington et al. [24]. GloVe embeddings are trained using global word co-occurrence statistics over large datasets, effectively combining global and local statistics for semantic modeling.
- **BERT-based models:** the most prominent BERT-based models for embedding generation are Sentence-BERT (SBERT) [25] and RoBERTa [20], both optimized extensions of the original BERT<sup>4</sup> architecture [7]. Both improve BERT's capability for similarity tasks. This adaptation significantly boosts their performance in applications such as semantic search, question answering, and clustering.
- **Transformer-based models:** modern models like `text-embedding-3-small` or `text-embedding-3-large` by OpenAI,<sup>5</sup> `NV-Embed-v2`<sup>6</sup> by NVIDIA [18], or `meta-llama/LLaMA-3.1` by Meta<sup>7</sup> are highly effective embeddings due to their attention mechanisms [29] and pre-training on massive corpora.

These embeddings serve as the foundation for retrieval, as they allow the system to represent text chunks numerically, transforming semantic relationships into mathematical comparisons.

### 7.2.1.3 Indexing

After generating embeddings, the next step is indexing these vector representations into a system optimized for efficient retrieval. Indexing organizes the vectors in a structured and searchable format, enabling the system to swiftly match incoming

<sup>3</sup> A dense vector is a numerical representation of data where most or all dimensions contain non-zero values.

<sup>4</sup> Bidirectional Encoder Representations from Transformers.

<sup>5</sup> <https://platform.openai.com/docs/guides/embeddings/embedding-models>.

<sup>6</sup> <https://huggingface.co/nvidia/NV-Embed-v2>.

<sup>7</sup> <https://huggingface.co/meta-llama>.

queries with stored data. This process typically involves selecting between *vector libraries* and *vector databases*, each tailored to specific use cases:

- **Vector libraries:** these systems are primarily used for static datasets where the indexed data remains immutable. Vector libraries focus solely on storing vector embeddings, without preserving the associated objects they were generated from. This approach means they lack support for CRUD (Create, Read, Update, Delete) operations. Examples of popular vector libraries include FAISS,<sup>8</sup> which is widely adopted for its efficiency in static and large-scale datasets; Annoy,<sup>9</sup> developed by Spotify and quite similar to FAISS since it is designed to work with static data; or ScaNN<sup>10</sup> by Google, designed for fast approximate nearest neighbor searches in high-dimensional spaces.
- **Vector database:** designed for dynamic and scalable applications, dedicated vector databases offer native support for vector operations and the associated data objects. These systems enable advanced functionalities such as CRUD operations, metadata integration, and real-time updates. Examples include Pinecone,<sup>11</sup> Weaviate,<sup>12</sup> and Milvus.<sup>13</sup> Such databases are particularly suitable for use cases requiring frequent updates or where contextual information must accompany the vectors for refined retrieval.

Efficient indexing ensures that the retrieval system can quickly compute similarity scores between incoming queries and stored vectors. The choice between a vector library and a vector database depends on the application's requirements, such as data mutability, scalability, and the need for enhanced metadata handling.

### 7.2.2 Retrieving Phase

Once embeddings are indexed, the retrieval mechanism is activated during the query phase, which is central to the RAG framework. This process involves transforming a user query into a format that can be effectively matched against the indexed embeddings to retrieve relevant context passages. Here's a detailed explanation of the retrieval phase:

- **Query encoding:** when a user issues a query, it is first encoded using the same embedding model that was used during the indexing phase. This ensures compatibility between the query's vector representation and the indexed embeddings. The

---

<sup>8</sup> Facebook AI Similarity Search: <https://github.com/facebookresearch/faiss/wiki>.

<sup>9</sup> Approximate Nearest Neighbors Oh Yeah: <https://github.com/spotify/annoy>.

<sup>10</sup> Scalable Nearest Neighbors: <https://github.com/google-research/google-research/tree/master/scann>.

<sup>11</sup> <https://github.com/pinecone-io>.

<sup>12</sup> <https://github.com/weaviate/weaviate>.

<sup>13</sup> <https://github.com/milvus-io/milvus>.

encoding step transforms the query into a dense vector that captures its semantic meaning.

- **Similarity computation:** the encoded query vector is compared against the stored embeddings in the vector database or library. This comparison is performed using similarity metrics, such as:
  - *Cosine similarity*, which measures the cosine of the angle between two vectors in multidimensional space
  - *Euclidean distance*, a measure for the straight-line distance between two vectors in space, suitable for certain use cases where magnitude<sup>14</sup> is important
  - *Dot product*, which computes the projection of one vector onto another, often optimized for speed in high-dimensional spaces
- **Result ranking:** the similarity scores computed in the previous step are used to rank the indexed embeddings. The system identifies the top- $K$  embeddings, those with the highest similarity scores, corresponding to the most relevant chunks of data.
- **Passage retrieval:** the top-ranked embeddings are then mapped back to their original text chunks (passages). Depending on the indexing system, additional metadata (e.g., document IDs, timestamps, or source information) can also be retrieved to enrich the results.

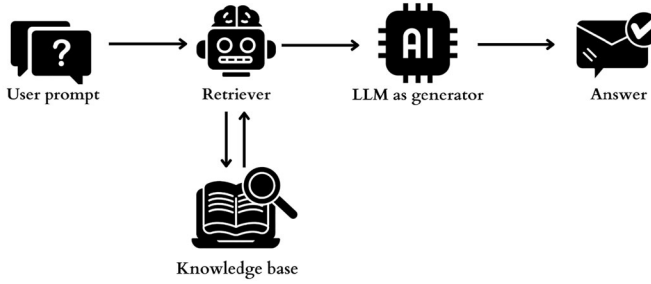
### 7.2.3 Generation Phase

After the retrieval process, the selected context (retrieved passages) is combined with the user query to form a prompt for the generative language model. This prompt serves as the input to the LLM, conditioning it on both the user query and retrieved knowledge. The model can then generate a response that incorporates this external context. The model's response is influenced by task-specific factors. It can use either (1) *parametric knowledge*, namely, inherent general knowledge from the model's pre-trained parameters, or (2) *non-parametric information*, which is information explicitly retrieved from external context. Figure 7.3 shows how the typical RAG framework works.

---

<sup>14</sup> *Magnitude* is the length of a dense vector, measuring its scale in the embedding space. It is crucial for normalization and similarity metrics, ensuring accurate matching between query vectors and stored data.





**Fig. 7.3** Typical RAG framework

### 7.2.4 RAG Classification

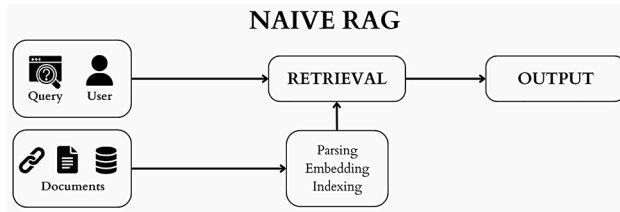
While the RAG paradigm is continuously evolving, researchers have proposed several categorizations based on multiple key components. Gao et al. [10] present a valuable classification that divides RAG into three main framework: *Naive RAG*, *Advanced RAG*, and *Modular RAG*. Additionally, RAG can be categorized based on the methodology employed by the retriever during the retrieval phase. Tonmoy et al. [27] propose three distinct RAG methodologies, specifically focusing on whether the retrieval occurs *before*, *during*, or *after* output generation. Zhao et al. [32] further extend this classification by categorizing RAG into four classes based on the retriever’s methodology to augment the generator: *Query-based RAG*, *Latent Representation-based RAG*, *Logit-based RAG*, and *Speculative RAG*.

In addition to these categorizations, another valuable classification arises from the type of knowledge source utilized in the RAG framework. RAG systems can be tailored to different knowledge sources, such as documents, knowledge graphs (KGs), and ontologies, each requiring distinct approaches for data retrieval and integration into the pipeline. The following subsections will go deeper into each stage of RAG, as well as explore the distinctions based on the type of information source employed.

#### 7.2.4.1 RAG Framework

Building on the categorization presented by Gao et al. [10], this paragraph aims to provide a thorough and detailed examination of the three framework of the RAG, *Naive RAG*, *Advanced RAG*, and *Modular RAG*.

**Naive RAG** The Naive RAG represents the earliest iteration of the RAG. It operates through a straightforward sequence of steps: parsing, embedding, indexing, retrieval, and generation. While Naive RAG provides a foundational framework for RAG systems, its simplicity comes with several limitations: (i) the retrieval phase often struggles with accuracy, frequently retrieving irrelevant or incorrect



**Fig. 7.4** Naive RAG framework

chunks and failing to capture all necessary information; *(ii)* during the generation phase, the model may produce hallucinations, generating content unsupported by the retrieved context. Additionally, *(iii)* responses can suffer from irrelevance, bias, or outright errors; integrating the retrieved information with the task at hand can be challenging, sometimes resulting in inconsistent or unclear outputs. Redundancy is another prevalent issue, as similar chunks may be retrieved multiple times, leading to repetitive responses. In Fig. 7.4, the Naive RAG framework with its steps is presented.

**Advanced RAG** Advanced RAG is designed to overcome the limitations of Naive RAG. The strategy adopted in this framework introduces two new processes, namely, *pre-retrieval* and *post-retrieval*, to optimize both indexing and query handling.

The *pre-retrieval* process focuses on two key elements: index optimization and query optimization. The goal of index optimization is to make the content of indexes more precise and representative to enhance retrieval accuracy. Techniques for this include improving data granularity (e.g., breaking down text into smaller, refined chunks for better management) and using mixed retrieval strategies that combine various types of data from multiple sources. On the other hand, query optimization involves the following processes: (i) *Query routing*, controlling which RAG flow the query is directed to; (ii) *Query rewriting*, rephrasing the query given in input from the user; and (iii) *Query expansion*, expanding the query with related terms to widen the search;

The *post-retrieval process* involves an initial re-ranking phase of the retrieved chunks to move the most relevant information to the top of the list provided to the LLM. This step is essential for prioritizing key information and generating more accurate outputs. Subsequently, a filtering step is performed: information is condensed through summarization and identification of critical sections to avoid overloading and help the model focus on essential and relevant details. Figure 7.5 illustrates the Advanced RAG framework.

**Modular RAG** The final framework in the evolution of the RAG is the *Modular RAG* [11], a structured, multi-level architecture designed to handle the increasing complexity of modern information retrieval and generation tasks. This architecture is organized across three distinct levels, each responsible for enhancing system modularity, scalability, and efficiency.

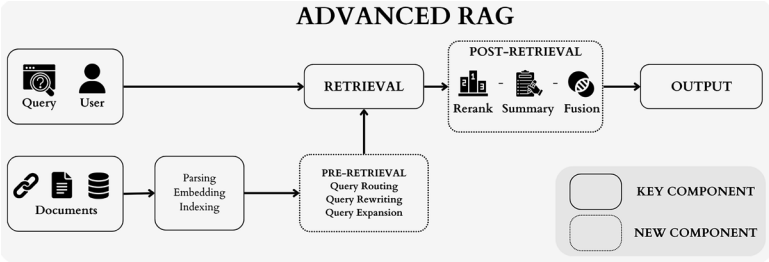


Fig. 7.5 Advanced RAG framework

**Primary Level** The framework follows a structured pipeline consisting of the five main stages presented in advanced RAG: parsing, embedding, and indexing, pre-retrieval, retrieval, post-retrieval, and generation. Each stage is responsible for a specific phase in the information processing workflow, ensuring a systematic progression from data preparation to content generation.

**Secondary Level** In this level, each stage of the pipeline is decomposed into specialized modules that handle distinct sub-tasks. For example, within the first step of pipeline (parsing, embedding, and indexing), modules like *chunk optimization* and *structural organization* ensure that data is processed and structured efficiently for retrieval. In pre-retrieval, components such as *query transformation*, *query expansion*, and *query construction* (as it has been already mentioned in advanced RAG explanation) refine the input query to improve retrieval accuracy. The retrieval stage integrates advanced mechanisms like *retriever fine-tuning*, *retriever source*, and *retriever selection*, enabling fine-grained control over data retrieval strategies. Similarly, post-retrieval involves modules for *reranking*, *compression*, and *selection*, optimizing the relevance and efficiency of retrieved information before generation. Finally, the generation phase incorporates *generator fine-tuning* and *verification* to ensure high-quality, accurate outputs.

**Third Level** This is the orchestration level in which the framework coordinates the interactions between modules through adaptive control mechanisms that dynamically govern the RAG process. Unlike rigid, linear approaches, Modular RAG introduces decision-making at pivotal junctures, allowing for flexible *routing*, *scheduling*, and *knowledge-guided reasoning*. (i) Routing directs queries through specific pipelines based on the input’s characteristics, using both metadata and semantic information to determine the most appropriate modules for each scenario. This decision-making process involves assigning scores to potential routes, enabling the system to activate only the most relevant modules for a given query. (ii) Scheduling manages the execution flow, identifying critical moments where additional retrieval or processing might be necessary. This mechanism supports recursive, iterative, and adaptive retrieval, ensuring that the system dynamically adjusts its operations based on the quality of intermediate results. (iii) The knowledge guide integrates knowledge graphs to construct reasoning chains that guide retrieval and

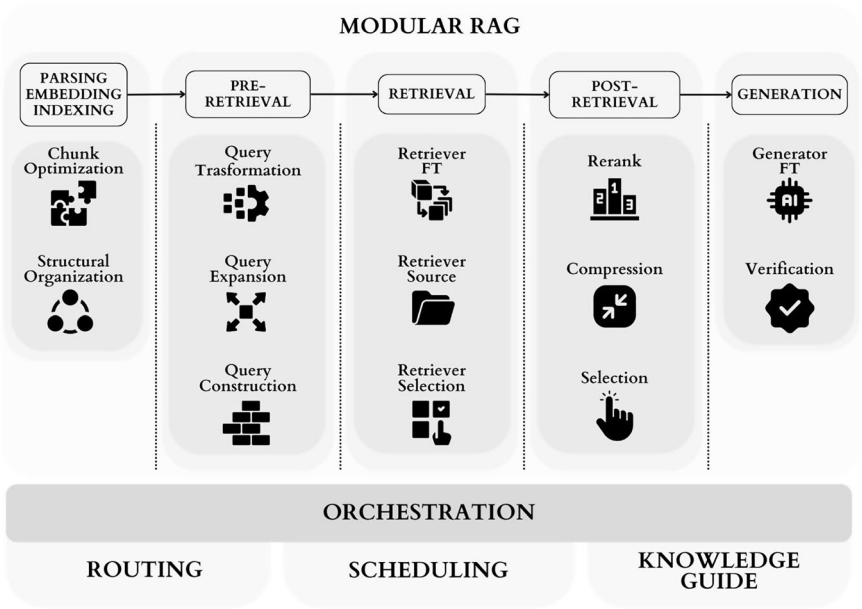


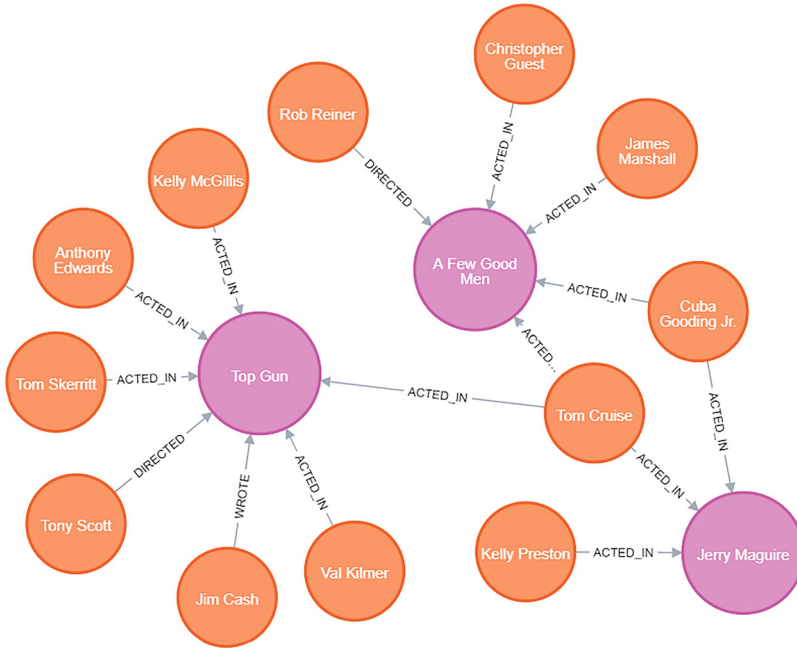
Fig. 7.6 Modular RAG framework

generation tasks. This part will be better explained in the next section since RAG across different knowledge sources will be discussed. In Fig. 7.6 modular RAG framework is presented.

7.2.4.2 RAG Across Knowledge Sources

Having established a detailed understanding of the fundamental workings of RAG, these paragraphs shift focus to explore how the framework can be adapted to different types of knowledge sources. The emphasis will be on demonstrating how the same foundational principles previously presented can be extended and customized to interact with various data formats to tackle different linguistic tasks. Specifically, we will focus on Classical RAG, RAG with KG, and RAG with ontologies. While no further details on knowledge sources are necessary for Classic RAG, as it is based on documents in various formats, it is essential to briefly explain what a knowledge graph and an ontology are.

According to [14], a KG is a *graph of data intended to accumulate and convey knowledge of the real world, whose nodes represent entities of interest and whose edges represent potentially different relations between these entities*. Given this definition, it is clear that KG represents information as a network of entities (nodes)



**Fig. 7.7** An example of generic KG of people (actors and directors) who directed or acted in films. This KG is extracted from the default MOVIE DB within neo4j

and their relationships (edges), specifically as triplets *subject-predicate-object*. An example of KG created using Neo4j<sup>15</sup> is presented in Fig. 7.7.

The strengths of knowledge graphs are several. Firstly, their *semantic structure* enables the embedding of explicit relationships, making them more intuitive for both human and machine reasoning [8]. Additionally, KGs offer significant *interconnectivity*, as an entity can have multiple connections to other entities and/or concepts, facilitating seamless access to both direct and indirect relationships. This structure allows for the exploration of first- and second-level links, providing a more comprehensive understanding of the data [2]. Finally, KGs are highly *scalable* [15], as they can represent knowledge at varying levels of granularity. They range from global knowledge bases like Wikidata<sup>16</sup> or DBpedia<sup>17</sup> to highly specialized, domain-specific graphs.

<sup>15</sup> Neo4j is a native graph database platform designed to store and query connected data using a graph-based model of nodes, relationships, and properties, optimized for analyzing complex interconnections: <https://neo4j.com/>.

<sup>16</sup> Wikidata [23] is a free, collaborative, multilingual knowledge graph: [https://www.wikidata.org/wiki/Wikidata:Main\\_Page](https://www.wikidata.org/wiki/Wikidata:Main_Page).

<sup>17</sup> DBpedia [1] is a project that extracts structured content from Wikipedia, transforming it into a semantic knowledge base: <https://www.dbpedia.org/>.



*theory*, with a relationship that links them through a more abstract concept, such as *discovered*, but also adding logical rules such as *if someone is a scientist, then they are a person*. This allows the system to make inferences and reason about the relationships, offering a deeper, more structured understanding of the concepts involved. Ontologies are thus particularly useful in contexts that require logical reasoning and inference, such as in knowledge management systems or AI applications.

**Classical RAG** Classic RAG is the foundational form of RAG, leveraging a retriever and generator architecture to process and synthesize information from unstructured text documents. These documents can come in various formats, such as PDFs, Word files, HTML pages, or plain text, and do not require any predefined schema or structural organization. The system's versatility lies in its ability to handle diverse data formats without prior restructuring, making it a straightforward approach suitable for a wide range of applications. This simplicity, however, may limit its effectiveness in scenarios requiring complex reasoning or deeper integration of structured knowledge.

**RAG with KG** RAG with KG builds upon the classic RAG approach by incorporating structured knowledge into the retrieval process. In RAG-KG, the retriever operates on the graph structure rather than unstructured text. Queries are processed to identify relevant entities or subgraphs that provide context-rich and highly structured information. The retrieved subgraph or triples (subject-predicate-object) are then used to enhance the generation phase.

**RAG with Ontologies** RAG with ontologies is an innovative extension of the traditional RAG paradigm, where the knowledge base used for retrieval and generation is structured as an ontology [6, 31]. Unlike classic RAG models that rely on unstructured documents or KG, RAG with ontologies takes advantage of the formal, logically structured nature of ontologies to enable more precise, context-aware reasoning and retrieval.

### 7.3 RAG Applications

This section transitions into practical implementations. The first focus will be on Python-based implementations of RAG pipelines, specifically on Classic RAG, RAG with KG, and RAG with ontologies, utilizing two prominent libraries: LangChain<sup>20</sup> and LlamaIndex.<sup>21</sup> These tools provide comprehensive frameworks for integrating retrieval and generation components, simplifying the construction

---

<sup>20</sup> <https://www.langchain.com/>.

<sup>21</sup> <https://www.llamaindex.ai/>.

of scalable and effective RAG systems. The second regards framework for RAG evaluation, specifically DeepEval and RAGAS.

By demonstrating how to leverage these libraries, we aim to equip practitioners with the practical knowledge needed to build customized RAG pipelines tailored to specific use cases.

### 7.3.1 Practical Application—LlamaIndex

This first hands-on guide provides a detailed walk-through of how to implement the aforementioned RAG pipelines using LlamaIndex. Thanks to the flexible functionalities offered by this framework, the import, configuration, and final query processes remain identical across all three RAG types. However, the core processes of parsing, embedding, and indexing differ for each type. This design enables a perfectly modular architecture where the central components of the code can be seamlessly swapped to transition from one RAG type to another.

#### Import

```
1 import openai
2 import os
3 import fitz
4 import nest_asyncio
5
6 from rdflib import Graph
7 from dotenv import load_dotenv
8
9 from llama_index.llms.openai import OpenAI
10 from llama_index.core.settings import Settings
11 from llama_index.core import (
12     Document,
13     VectorStoreIndex,
14     PropertyGraphIndex,
15     SimpleDirectoryReader
16 )
```

This first block of code sets up the necessary libraries and configurations for implementing a RAG pipeline. It uses LlamaIndex for managing document retrieval and indexing. Key components from LlamaIndex are `VectorStoreIndex` for storing and querying vector embeddings, `PropertyGraphIndex` for constructing property graphs from the document's data, and `SimpleDirectoryReader` for file loading. Additionally, the code integrates OpenAI models for the generation process, `Settings` to configure global settings for language models, embeddings, and other behaviors and uses `dotenv` for securely managing environment variables like API keys. `nest_asyncio` enables nested asynchronous event loops, very



useful in interactive environments where an event loop is already running in the background, like in Jupyter Notebooks.

### Configuration

```
1 # Apply the asyncio patch to allow nested event loops.
2 nest_asyncio.apply()
3
4 # Load environment variables from a .env file.
5 load_dotenv()
6
7 # Retrieve the OpenAI and Llama API key from the environment variables.
8 openai_api_key = os.getenv("OPENAI_API_KEY")
```

This block of code securely manages sensitive information, such as OpenAI API key<sup>22</sup> (line 8). The `load_dotenv()` function (line 5) reads environment variables from a `.env` file located in the same directory as the script. This practice keeps sensitive information separate from the code, minimizing the risk of exposure in version control systems or public repositories. Environment variables can then be accessed using `os.getenv()` and set with `os.environ`, ensuring their security and anonymity. For proper functionality, ensure the **.env file is in the same directory as the script**.

### File loading

```
1 # Specify required extensions as a list (if needed)
2 required_exts = [] # .png, .jpg, .pdf, .md, .txt, .csv etc...
3
4 # Load documents from a specified directory
5 reader = SimpleDirectoryReader(
6     input_dir="path/to/your/directory",
7     input_files=["path/to/your/directory/document_1",
8                 "path/to/your/directory/document_2"],
9     required_exts=required_exts
10 )
11
12 # Load and parse document from the reader
13 document = reader.load_data(num_workers=4)
```

In this part, `SimpleDirectoryReader` function serves to load multiple documents from a specified directory. The `input_dir` parameter (line 6) specifies the main directory from which all documents will be loaded automatically. This is useful for batch processing files without manually listing each one. On the other

<sup>22</sup> To activate an API key for OpenAI, visit: <https://platform.openai.com/api-keys>.

hand, the `input_files` parameter (line 7) allows for specifying individual file paths directly, giving you the flexibility to load specific files from different locations if needed. Finally, the `required_exts` list (line 2) can be customized to filter files based on their extensions, such as `.pdf`, `.txt`, `.md`, or `.csv`. This setup enables the batch processing of multiple files simultaneously, making it ideal for preparing documents to populate a vector database. Finally, documents are loaded from the reader using `load_data` method (line 13).

These first blocks of code (import, configuration, and file loading) can be used as the basis for each following RAG type.

### 7.3.1.1 Classic RAG

#### Classic RAG - Pipeline set-up

```

1  # Create a vector index from the parsed documents.
2  index = VectorStoreIndex.from_documents(document)
3
4  # Initialize the OpenAI language model (LLM) for generating responses.
5  llm = OpenAI(
6  model="gpt-4-turbo",
7  strict=True,           # True default
8  temperature=0,        # from 0 to 1
9  max_tokens=2000,      # from 0 to infinity (None default)
10 top_p=0.8)             # from 0 to 1
11
12 # Convert the vector index into a query engine for running queries on
   ↳ the indexed documents.
13 query_engine = index.as_query_engine(
14 llm=llm,
15 response_mode = "compact",
16 streaming = True,
17 similarity_top_k=2,
18 verbose=True)

```

In this core part of the RAG pipeline, the process begins with the indexing of the document using `VectorStoreIndex` (line 2), which organizes the content into vector embeddings. These embeddings enable efficient similarity-based retrieval. Next, the OpenAI language model is initialized with the `OpenAI` function (lines 5). The parameters are as follows: `strict=True` ensures that input data is strictly validated to conform to expected formats; `temperature=0` is set to generate deterministic responses, reducing randomness for more consistent results; `max_tokens=2000` allows the model to generate longer responses when needed, up to a maximum of 2000 tokens; and `top_p=0.8` controls the diversity of the responses, narrowing down the possible options while maintaining some variety. Finally, a query engine is created using the `as_query_engine` method (line 13). The `verbose=True` option

enables detailed logging, providing real-time information about the execution and process status.

### 7.3.1.2 RAG with KG

#### RAG with KG - Pipeline set-up

```

1  # Create a PropertyGraphIndex from the parsed documents.
2  index = PropertyGraphIndex.from_documents(document)
3
4  # Initialize the OpenAI language model (LLM) for generating responses.
5  llm = OpenAI(
6  model="gpt-4-turbo",
7  strict=True,          # True default
8  temperature=0,        # from 0 to 1
9  max_tokens=2000,      # from 0 to infinity (None default)
10 top_p=0.8)            # from 0 to 1
11
12 # Convert the vector index into a query engine for running queries on
13 ↪ the indexed documents.
14 query_engine = index.as_query_engine(
15 llm=llm,
16 response_mode = "compact",
17 streaming = True,
18 similarity_top_k=2,
19 verbose=True)

```

This further block represents the core of RAG with KG. First, the documents are indexed using `PropertyGraphIndex` (line 2), a method tailored for constructing property graphs from the document's data. This indexing approach supports the representation of relationships and metadata, enhancing the system's ability to query and analyze structured knowledge. Finally, `llm` with `OpenAI` function (line 5) and a query engine with `query_engine` (line 13) are configured using `as_query_engine`. Key parameters are the same for Classic RAG.

### 7.3.1.3 RAG with Ontologies

#### RAG with Ontologies - Pipeline set-up

```

1  # Initialize an RDF graph and parse the selected XML file into it.
2  g = Graph()
3  g.parse(str(reader.input_files[0]), format="xml") # must be .owl, .json
4  ↪ or .csv
5
6  # Create a list of nodes from the RDF graph
7  nodes = []

```

```

7 documents_ontology = []
8
9 for s, p, o in g:
10 nodes.append({
11     "subject": str(s),
12     "predicate": str(p),
13     "object": str(o), })
14
15 # Generate documents from RDF nodes and store them in the
16 ⇨ documents_ontology list.
17 for node in nodes:
18 content = f"Subject: {node['subject']}, Predicate: {node['predicate']},
19 ⇨ Object: {node['object']}"
20 documents_ontology.append(Document(text=content))
21
22 # Index the documents into a PropertyGraphIndex for semantic search.
23 index_ontology = PropertyGraphIndex.from_documents(documents_ontology)
24
25 # Initialize the OpenAI model with specific configurations for response
26 ⇨ generation.
27 llm = OpenAI(
28 model="gpt-4-turbo",
29 strict=True, # True default
30 temperature=0, # from 0 to 1
31 max_tokens=2000, # from 0 to infinity (None default)
32 top_p=0.8), # from 0 to 1
33
34 # Set up the query engine to perform semantic search on ontology index.
35 query_engine = index_ontology.as_query_engine(
36 llm=Settings.llm,
37 response_mode = "compact",
38 streaming = True,
39 similarity_top_k=2,
40 verbose=True)

```

This final block of code illustrates the core implementation of an ontology-based RAG pipeline, which uses RDF data to construct a retrieval system through a property graph. Initially, the code leverages the RDF graph parser `Graph()` (lines 2–3) to load and parse the selected ontology file. The ontology must be provided in `.owl`, `.json`, or `.csv` format, with the data being processed as XML. The RDF triples are then iterated through, and each triple is stored as a structured node, where each node captures a distinct relationship or fact derived from the ontology. Next, these nodes are converted into text documents, stored in the list `documents_ontology` (line 18). Each document represents an RDF triple as a textual string, ensuring the data is suitable for downstream indexing and retrieval. These textual representations are indexed to create a `PropertyGraphIndex` (line 21), which organizes the ontology data into a graph structure. The `llm` is set with `OpenAI` function (line 24). Finally, a query engine using `query_engine` (line 32) is instantiated using the property graph. The `llm` and the engine are configured with

parameters previously discussed, enabling it to process user queries by retrieving relevant results based on the indexed ontology data.

#### 7.3.1.4 Query and Response

##### Query and response

```
1 user_query = input("Enter your query: ")
2 response = query_engine.query(user_query)
```

This final segment of the pipeline is designed to execute user queries across **any of the RAG configurations previously discussed**. The `user_query` (line 1) serves as the input for the query engine (line 2), which processes the query and retrieves relevant context from the indexed data to generate a response. The retrieval and response generation process is entirely interchangeable and adapts seamlessly to the underlying knowledge source. Additionally, the `user_query` can be enriched using prompt engineering techniques. By carefully structuring the query with specific instructions, constraints, or context, the system can be better directed toward performing the desired task, such as summarization, reasoning, or detailed explanation.

### 7.3.2 Practical Application—Langchain

This second hands-on guide focuses on implementing RAG pipelines using the LangChain framework. Unlike the previous section, where we provided plug-and-play code structures applicable to all three types of RAG (classical, RAG with knowledge graphs, and RAG with ontologies), the implementations here are more specialized. While the initial steps for classical RAG and ontology RAG can be unified, implementing RAG with KG requires a separate block of code.

#### 7.3.2.1 Classic RAG and Ontology RAG

##### Import

```
1 import os
2 import json
3 import nest_asyncio
4
5 from dotenv import load_dotenv
6
7 from langchain.text_splitter import RecursiveCharacterTextSplitter
```

```

8 from langchain.vectorstores import FAISS
9 from langchain.chains import RetrievalQA
10 from langchain_openai import ChatOpenAI, OpenAIEmbeddings
11 from langchain.document_loaders import (
12     TextLoader,
13     PyPDFLoader,
14     CSVLoader,
15     JSONLoader
16 )

```

This first block of code sets up the necessary libraries and configurations for implementing a RAG pipeline using Langchain. It imports various modules for document loading, such as `TextLoader`, `PyPDFLoader`, `CSVLoader`, and `JSONLoader` (lines 11–15), which handle different file formats (.txt, .pdf, .csv, .json). The `RecursiveCharacterTextSplitter` (line 7) is used to split large documents into smaller chunks for easier processing and querying. The code utilizes `FAISS` (line 8) for efficient vector storage and similarity-based retrieval of document embeddings. For the generation process, the `ChatOpenAI` model and `OpenAIEmbeddings` (line 10) are integrated to facilitate semantic search and response generation. The purpose of `dotenv` and `nest_asyncio` libraries have already been explained in previous section.

### Configuration

```

1 # Apply the asyncio patch to allow nested event loops.
2 nest_asyncio.apply()
3
4 # Load environment variables from a .env file.
5 load_dotenv()
6
7 # Retrieve the OpenAI API key from the environment variables.
8 openai_api_key = os.getenv("OPENAI_API_KEY")

```

Since this is identical to the previous implementation, no further explanation is needed.

### File Loading

```

1 # Load a PDF document from the specified path using PyPDFLoader.
2 loader = PyPDFLoader('path/to/your/directory/document.pdf')
3 # loader = TextLoader('path/to/your/directory/document.txt')
4 # loader = CSVLoader('path/to/your/directory/document.csv')
5 # loader = JSONLoader('path/to/your/directory/document.json')
6
7 document = loader.load()

```

This simple block of code manages file selection and document loading for the RAG pipeline. Specifically, it loads PDF files using the `PyPDFLoader` function (line 2). If a different document format needs to be loaded, this function can be replaced with `TextLoader` for `.txt` files, `CSVLoader` for `.csv` files, or `JSONLoader` for `.json` files, as presented in comments (lines 3–5).

### Parsing, Embedding, Indexing

```

1  # Split documents into smaller chunks for easier processing.
2  text_splitter = RecursiveCharacterTextSplitter(
3      chunk_size=1000,
4      chunk_overlap=200
5  )
6
7  # Split the documents into smaller chunks using the text splitter.
8  docs = text_splitter.split_documents(document)
9
10 # Create embeddings using the "text-embedding-3-large" model
11 embeddings = OpenAIEmbeddings(model="text-embedding-3-large")
12
13 # Create a FAISS vector store from the documents and embeddings for
14 # ↪ efficient similarity search.
15 vectorstore = FAISS.from_documents(docs, embeddings)
16
17 # Initialize the ChatOpenAI model with specific configurations for
18 # ↪ response generation.
19 llm = ChatOpenAI(
20     model="gpt-4o",
21     temperature=0,          # from 0 to 1
22     max_tokens=2000,        # from 0 to infinity
23     top_p=0.8,              # from 0 to 1
24     api_key=openai_api_key)
25
26 # Set up a RetrievalQA chain for querying
27 qa_chain = RetrievalQA.from_llm(
28     llm=llm,
29     retriever=vectorstore.as_retriever(),
30 )

```

This block of code sets up the core components of the RAG pipeline. The `RecursiveCharacterTextSplitter` (line 2) is initialized with a chunk size of 1000 characters and an overlap of 200 characters, ensuring that the documents are split into manageable and contextually cohesive chunks. Note that `chunk_overlap` must be smaller than `chunk_size`. The `split_documents` method (line 8) is then used to apply this splitter to the loaded documents, effectively preparing them for downstream tasks. To handle embeddings, the `OpenAIEmbeddings` class (line 11) is instantiated using the `text-embedding-3-large` model. Other available embedding models include `text-embedding-3-small` and

text-embedding-ada-002.<sup>23</sup> These embeddings are stored in a FAISS vector store via the `FAISS.from_documents` method (line 14), enabling efficient similarity-based searches. The `ChatOpenAI` model (line 17) is then initialized with specific parameters, the same presented for `LlamaIndex` in the previous section. Finally, the `RetrievalQA.from_llm` method (line 25) creates a retrieval-based question-answering chain by linking the initialized language model (`llm`) and the FAISS vector store retriever (`vectorstore.as_retriever()`).

#### Query and response

```
1 user_query = input("Enter your query: ")
2 response = qa_chain.invoke(user_query)
```

In this final block the user query is processed using the `qa_chain.invoke` method (line 3). The functionality mirrors the equivalent steps in the `LlamaIndex` framework.

### 7.3.2.2 RAG with KG

Now we present an alternative implementation of a RAG pipeline with knowledge graphs.

#### Import

```
1 import os
2
3 from dotenv import load_dotenv
4 from neo4j import GraphDatabase
5
6 from langchain_openai import ChatOpenAI
7 from langchain.chains import GraphCypherQAChain
8 from langchain.graphs import Neo4jGraph
9 from langchain.prompts import PromptTemplate
```

This initial block imports the required libraries for integrating a knowledge graph with the RAG pipeline. It includes `Neo4j` as the graph database backend and `LangChain` modules such as `GraphCypherQAChain` and `Neo4jGraph` for

<sup>23</sup> More information on embedding models by OpenAI is available at <https://platform.openai.com/docs/guides/embeddings>.



interacting with the KG. The `PromptTemplate` is used to customize queries sent to the model, while `pprint` assists with structured output during debugging.

### Configuration and KG loading

```

1  # Load environment variables from a .env file
2  load_dotenv()
3
4  # Retrieve API keys and database credentials from a .env file
5  openai_api_key = os.getenv("OPENAI_API_KEY")
6  neo4j_username = os.getenv("NEO4J_USERNAME")
7  neo4j_password = os.getenv("NEO4J_PASSWORD")
8
9  # Retrieve the Neo4j database URI and set authentication credentials.
10 URI = os.getenv("NEO4J_URI")
11 AUTH = (neo4j_username, neo4j_password)
12
13 # Initialize the Neo4jGraph connection with enhanced schema support.
14 graph = Neo4jGraph(
15     url=URI,
16     username=neo4j_username,
17     password=neo4j_password,
18     enhanced_schema=True,
19 )
20
21 # Define a Cypher query to load CSV data from a specific URL and create
22 ↪ nodes/relationships.
23 cypher_query = """
24 LOAD CSV WITH HEADERS FROM 'specific URL to load CSV file' AS row
25
26 // Replace the following MERGE clauses with your specific data modeling
27 ↪ logic.
28 // Use MERGE to create nodes and relationships based on the data in the
29 ↪ CSV.
30 // Add your domain-specific MERGE clauses here
31 """
32 graph.query(cypher_query)

```

This block configures environment variables for connecting to the Neo4j database and OpenAI services. Using `dotenv`, it retrieves and sets the necessary details, including the OpenAI API key, Neo4j URI, username, and password from the `.env` file (lines 5–11). Following this, the `Neo4jGraph` class (line 14) from `Langchain` is initialized using the Neo4j URI, username, and password, which are retrieved from the environment variables. This object provides a convenient interface to interact with the graph database. The `enhanced_schema` parameter (line 18) is set to `True` to enable advanced schema capabilities for more complex graph queries. The first part of the code allows you to interact with and upload an existing graph that is already present in Neo4j. The second part of the code allows you to execute a Cypher query (line 22) that performs an upload from a CSV file

into the Neo4j graph database. The `LOAD CSV` clause in Cypher loads data from the specified file, and the `MERGE` clauses within the query are used to create nodes and relationships based on the contents of the CSV file. The `insert specific URL to load CSV file placeholder` (line 23) should be replaced with the actual location of the CSV file to be uploaded.

### Pipeline set-up

```

1  # Define a custom prompt template for user input, allowing future
   ↳ customization.
2  chat_prompt = PromptTemplate(
3  input_variables=["user_input"],
4  template="" # Template left intentionally blank for customization
5  )
6
7  # Initialize the OpenAI model with specific configurations for response
   ↳ generation
8  llm = ChatOpenAI(
9  model="gpt-4o",
10 temperature=0,           # from 0 to 1
11 max_tokens=2000,         # from 0 to infinity
12 top_p=0.8,              # from 0 to 1
13 api_key=openai_api_key)
14
15 # Create a GraphCypherQAChain for querying the Neo4j graph using
   ↳ natural language.
16 chain = GraphCypherQAChain.from_llm(
17 llm,
18 graph=graph,
19 prompt=chat_prompt,
20 verbose=True,
21 return_intermediate_steps=True,
22 validate_cypher=True,
23 top_k=5,
24 allow_dangerous_requests=True
25 )

```

This block of code sets up a `GraphCypherQAChain` (line 16), enabling queries on a Neo4j graph using OpenAI's `gpt-4o` model. A `PromptTemplate` defines the input format for user queries, allowing for future customization as needed. The `ChatOpenAI` model (line 2) is configured with key parameters such as `temperature`, `max_tokens`, and `top_p`, while the OpenAI API key is provided for authentication. The `GraphCypherQAChain` connects the language model, the Neo4j graph instance, and the input prompt (line 17–19). Additional options include enabling verbose logging for detailed output, returning intermediate reasoning steps, validating Cypher queries before execution, limiting

the number of results with `top_k`, and permitting potentially unsafe requests via `allow_dangerous_requests` (line 20–24).

#### Query and response

```
1 user_query = input("Enter your query: ")
2 response = chain.invoke(users_query)
```

Finally, we can execute our query using `chain.invoke` (line 2).

### 7.3.3 RAG Evaluation

Evaluating RAG systems is crucial for ensuring their effectiveness in real-world applications. Unlike traditional generative models, RAG combines retrieval and generation components, which requires evaluation strategies that assess both aspects comprehensively. Earlier assessments of RAG systems often focused on task-specific metrics, such as F1 and EM scores [28, 33] for Question Answering or Accuracy for Fact-Checking [26]. However, modern evaluation frameworks now emphasize not only task performance but also the quality of retrieval and the fidelity of generation, making them more suited to assess the unique characteristics of RAG systems.

This section introduces two widely used frameworks for RAG evaluation: DeepEval<sup>24</sup> and RAGAS.<sup>25</sup> First, we will present the key evaluation metrics, which are divided into *retrieval evaluation metrics* and *generation evaluation metrics*, to offer a comprehensive understanding of how to measure the quality of retrieval and generated outputs. In this context, it is important to clarify that all of these metrics use LLMs as a critical component for assessing the quality of the retrieval and generation processes. For retrieval metrics such as *Contextual Precision* and *Contextual Recall*, the LLM evaluates the relevance of nodes in the retrieval context and determines their alignment with the input and expected output. For generation metrics like *Answer Relevancy* and *Faithfulness*, the LLM evaluates the factual accuracy and relevance of the generated output in relation to the input query and retrieval context. Additionally, the LLM provides reasoning for its judgments, which enhances the transparency and interpretability of the evaluation process. After introducing these metrics, we will provide implementation examples for DeepEval, demonstrating how to apply them in practice. Due to space limitations, detailed implementation examples for RAGAS will be included in GitHub.

<sup>24</sup> <https://docs.confident-ai.com/docs/getting-started>.

<sup>25</sup> <https://docs.ragas.io/en/stable/>.

### 7.3.3.1 Retrieval Evaluation Metrics

The following retrieval evaluation metrics are essential for assessing the performance of the retriever component in a RAG pipeline. These metrics evaluate the quality and relevance of the retrieved documents or nodes from the retrieval context in relation to the expected output.

**Contextual Precision** The *Contextual Precision* metric (CP) assesses the effectiveness of the retriever by measuring how well the retrieved nodes in the retrieval context match the relevance of the given input query. In particular, it evaluates whether relevant nodes are ranked higher than irrelevant ones. This metric employs a self-explaining LLM-based evaluation, where an LLM is used to assess the relevance of each retrieved node to the query, and justifies its relevance judgment. The CP score prioritizes nodes that appear earlier in the retrieval list, reflecting their higher importance. The CP is calculated using the following formula:

$$CP = \frac{1}{No. \text{ of Relevant Nodes}} \cdot \sum_{k=1}^n \left( \frac{No. \text{ of Relevant Nodes up to Position } k}{k} \cdot r_k \right)$$

where:

- $k$  is the position of the  $k$ -th node in the *retrieval\_context*
- $n$  is the total number of nodes in the *retrieval\_context*
- $r_k$  is the binary relevance of the  $k$ -th node ( $r_k = 1$  if relevant,  $r_k = 0$  otherwise)

The resulting score is computed using the *Weighted Cumulative Precision* (WCP), which aggregates the relevance assessments while prioritizing nodes ranked higher in the retrieval list. The WCP increases the contribution of relevant nodes that appear earlier in the ranking, reflecting their greater importance in a retrieval context.

**Contextual Recall** The *Contextual Recall* metric (CR) evaluates the ability of the retriever to capture all the relevant information in the retrieval context with respect to the expected output. It measures how well the retrieval context aligns with the expected output by calculating the proportion of statements in the expected output that are supported by the retrieved context. Similar to CP, this metric also uses an LLM-based evaluation to assess how well each statement in the expected output is supported by the retrieval context. The CR is calculated as:

$$CR = \frac{Number \text{ of Attributable Statements}}{Total \text{ Number of Statements in Expected Output}}$$

where:

- *Attributable statements* are statements in the expected output that can be traced back to the nodes in the retrieval context

- *Total number of statements* refers to the total number of distinct statements identified in the expected output

The LLM plays a crucial role here by extracting and classifying the statements made in the expected output. It then determines which of those statements are attributable to the nodes in the retrieval context. The final score reflects the proportion of statements in the expected output that are adequately supported by the retrieved context.

### 7.3.3.2 Generation Evaluation Metrics

The following metrics focus on evaluating the quality of the generated output in a RAG pipeline, specifically assessing the coherence, relevance, and factual accuracy of the answers generated by the model.

**Answer Relevancy** The *Answer Relevancy* metric evaluates how relevant the actual output generated by the LLM is in relation to the input query. It measures the proportion of relevant statements in the generated output compared to the total number of statements. This metric is essential for understanding how well the generated answer addresses the user's query. The Answer Relevancy Metric (AR) is calculated as follows:

$$AR = \frac{\text{Number of Relevant Statements}}{\text{Total Number of Statements in Actual Output}}$$

where:

- *Relevant statements* are those statements in the actual output that are directly related to the input query
- *Total number of statements* refers to the total number of distinct statements made in the actual output

The LLM is responsible for evaluating the relevance of each statement in the generated output. It provides the reasoning behind its classification, enabling transparency in the evaluation process.

**Faithfulness** The *Faithfulness* metric evaluates the degree to which the actual output aligns with the factual information contained in the retrieval context. It ensures that the generated answer does not introduce hallucinations or incorrect information, maintaining consistency with the retrieved context. This metric uses an LLM-based evaluation, where the LLM checks if the claims made in the generated output are truthful and supported by the retrieval context. The Faithfulness Metric (F) is calculated as:

$$F = \frac{\text{Number of Truthful Claims}}{\text{Total Number of Claims in Actual Output}}$$

where:

- *Truthful claims* are those claims in the actual output that are factually accurate and supported by the retrieval context
- *Total number of claims* refers to the total number of distinct claims made in the actual output

The LLM is used to verify the truthfulness of each claim in the generated output by comparing it to the information in the retrieval context. The reasoning behind the evaluation is provided, allowing for an explanation of which claims are considered truthful and which are not.

### 7.3.3.3 Practical Implementation with DeepEval

#### DeepEval - Import

```

1 # Importing the necessary evaluation metrics from the deepeval library
2 from deepeval.metrics import ContextualPrecisionMetric
3 from deepeval.metrics import ContextualRecallMetric
4 from deepeval.metrics import AnswerRelevancyMetric
5 from deepeval.metrics import FaithfulnessMetric
6 from deepeval.test_case import LLMTestCase
7 from deepeval import evaluate

```

In this first block of code, necessary library are imported, including those for the retrieval and generation evaluation metrics, namely, *Contextual Precision*, *Contextual Recall*, *Answer Relevancy*, and *Faithfulness* (lines 2–5). *LLMTestCase* and *evaluate* (lines 6–7) are fundamental for the actual execution of the evaluation process.

#### Metrics configuration and initialization

```

1 # Defining the environment variable for OpenAI API key
2 os.environ["OPENAI_API_KEY"] = "your_openai_api_key"
3
4 #Initializing test inputs
5 actual_output = " "
6 expected_output = " "
7 retrieval_context = []
8
9 # Initializing the ContextualPrecisionMetric
10 metric_CP = ContextualPrecisionMetric(
11     threshold = 0.7,
12     model = "gpt-4o-mini",
13     include_reason = True
14 )
15
16 # Initializing the ContextualRecallMetric
17 metric_CR = ContextualRecallMetric(

```

```

18 threshold = 0.7,
19 model = "gpt-4o-mini",
20 include_reason = True
21 )
22
23 # Initializing the AnswerRelevancyMetric
24 metric_AR = AnswerRelevancyMetric(
25     threshold = 0.7,
26     model = "gpt-4o-mini",
27     include_reason = True
28 )
29
30 # Initializing the FaithfulnessMetric
31 metric_F = FaithfulnessMetric(
32     threshold = 0.7,
33     model = "gpt-4o-mini",
34     include_reason = True
35 )

```

Here, the code sets up the api key (line 2) and the inputs for evaluation (lines 5–7). The `actual_output` (line 3) represents the generated response from the model, while the `expected_output` (line 4) is the ground truth or reference answer. The `retrieval_context` (line 5) is a list of strings retrieved by the system, which the model uses to generate its response. Moreover, all single metrics are initialized (lines 8, 15, 22, 29), setting specific parameters, such as a `threshold`, used to determine relevance, and the model to be evaluated, in this case, `gpt-4o-mini`. The `include_reason` parameter ensures that the reasoning behind the score is provided.

### Executing test

```

1 # Creating a test case to evaluate contextual precision (CP)
2 test_case_PC = LLMTestCase(
3     input = "",
4     actual_output = actual_output,
5     expected_output = expected_output,
6     retrieval_context = retrieval_context
7 )
8
9 # Creating a test case to evaluate contextual recall (CR)
10 test_case_CR = LLMTestCase(
11     input = "",
12     actual_output = actual_output,
13     expected_output = expected_output,
14     retrieval_context = retrieval_context
15 )
16
17 # Creating a test case to evaluate answer relevancy (AR)
18 test_case_AR = LLMTestCase(

```

```

19 input = "",
20 actual_output = actual_output,
21 )
22
23 # Creating a test case to evaluate faithfulness (F)
24 test_case_CR = LLMTestCase(
25     input = "",
26     actual_output = actual_output,
27     retrieval_context = retrieval_context
28 )
29
30 # Measuring the evaluation score for the test case
31 metric_CP.measure(test_case_CP)
32 metric_CP.measure(test_case_CR)
33 metric_CP.measure(test_case_AR)
34 metric_CP.measure(test_case_F)
35
36 # Evaluating the test case with the metric and displaying results
37 evaluate([test_case_CP], [metric_CP])
38 evaluate([test_case_CR], [metric_CR])
39 evaluate([test_case_AR], [metric_AR])
40 evaluate([test_case_F], [metric_F])

```

Inputs defined in *Metrics configuration and initialization* block go into each LLMTestCase function (lines 2, 10, 18, 24). The measure method (lines 31–34) is then called to evaluate the test case, calculating the score based on the provided inputs. Finally, the evaluation is conducted using the evaluate function (lines 37–40), which processes and outputs the final evaluation results.

## References

1. Auer, S., Bizer, C., Kobilarov, G., Lehmann, J., Cyganiak, R., Ives, Z.: Dbpedia: A nucleus for a web of open data. In: international Semantic Web Conference, pp. 722–735. Springer (2007)
2. Auer, S., Kovtun, V., Prinz, M., Kasprzik, A., Stocker, M., Vidal, M.E.: Towards a knowledge graph for science. In: Proceedings of the 8th International Conference on Web Intelligence, Mining and Semantics, pp. 1–6 (2018)
3. Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., Van Den Driessche, G.B., Lespiau, J.B., Damoc, B., Clark, A., et al.: Improving language models by retrieving from trillions of tokens. In: International Conference on Machine Learning, pp. 2206–2240. PMLR (2022)
4. Boykis, V.: What Are Embeddings. <https://doi.org/10.5281/zenodo.8015029> (2023)
5. Caballero, M.: A brief survey of question answering systems. Int. J. Artif. Intell. Appl. (IJAIA) **12**(5), (2021)
6. DeBellis, M., Duttav, N., Ginoc, J., Balajid, A.: Integrating ontologies and large language models to implement retrieval augmented generation (rag). Appl. Ontol. **1**, 1–5 (2024)
7. Devlin, J.: Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805 (2018)



8. Ehrlinger, L., Wöb, W.: Towards a definition of knowledge graphs. *SEMANTiCS (Posters, Demos, SuCCESS)* **48**(1-4), 2 (2016)
9. Ferrucci, D.A.: Introduction to “this is watson”. *IBM J. Res. Dev.* **56**(3.4), 1:1–1:15 (2012). <https://doi.org/10.1147/JRD.2012.2184356>
10. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., Wang, H.: Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* (2023)
11. Gao, Y., Xiong, Y., Wang, M., Wang, H.: Modular rag: Transforming rag systems into lego-like reconfigurable frameworks. *arXiv preprint arXiv:2407.21059* (2024)
12. Graham, K.: Techmatters: What happened to jeeves, you ask. com? *LOEX Quart.* **33**(3), 3 (2006)
13. Gruber, T.: What is an ontology (1993)
14. Hogan, A., Blomqvist, E.: Knowledge graphs. *ACM Comput. Surv. (Csur)* **54**(4), 1–37 (2021)
15. Ji, S., Pan, S., Cambria, E., Martinen, P., Philip, S.Y.: A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Trans. Neural Networks Learn. Syst.* **33**(2), 494–514 (2021)
16. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Madotto, A., Fung, P.: Survey of hallucination in natural language generation. *ACM Comput. Surv.* **55**(12), 1–38 (2023). <https://doi.org/10.1145/3571730>
17. Krajewski, M., Marcum Charles, I.: Ask Jeeves: Servants as search engines. *Grey Room* (38), 6–19 (2010). <https://doi.org/10.1162/grey.2010.1.38.6>
18. Lee, C., Roy, R., Xu, M., Raiman, J., Shoeybi, M., Catanzaro, B., Ping, W.: Nv-embed: Improved techniques for training llms as generalist embedding models. *arXiv preprint arXiv:2405.17428* (2024)
19. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., et al.: Retrieval-augmented generation for knowledge-intensive nlp tasks. *Adv. Neural Inf. Process. Syst.* **33**, 9459–9474 (2020)
20. Liu, Y.: Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* **364** (2019)
21. Mikolov, T.: Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781* **3781** (2013)
22. Noy, N.F., Crubézy, M., Fergerson, R.W., Knublauch, H., Tu, S.W., Vendetti, J., Musen, M.A.: Protégé-2000: an open-source ontology-development and knowledge-acquisition environment: Amia 2003 open source expo. In: *Amia Annual Symposium Proceedings*, vol. 2003, p. 953 (2003)
23. Pellissier Tanon, T., Vrandečić, D., Schaffert, S., Steiner, T., Pintscher, L.: From freebase to wikidata: The great migration. In: *Proceedings of the 25th International Conference on World Wide Web*, pp. 1419–1428 (2016)
24. Pennington, J., Socher, R., Manning, C.D.: Glove: Global vectors for word representation. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1532–1543 (2014)
25. Reimers, N.: Sentence-bert: Sentence embeddings using Siamese bert-networks. *arXiv preprint arXiv:1908.10084* (2019)
26. Shao, Z., Gong, Y., Shen, Y., Huang, M., Duan, N., Chen, W.: Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. *arXiv preprint arXiv:2305.15294* (2023)
27. Tonmoy, S., Zaman, S., Jain, V., Rani, A., Rawte, V., Chadha, A., Das, A.: A comprehensive survey of hallucination mitigation techniques in large language models. *arXiv preprint arXiv:2401.01313* (2024)
28. Van Rijsbergen, C.J.: Foundation of evaluation. *J. Document.* **30**(4), 365–373 (1974)
29. Vaswani, A.: Attention is all you need. *Adv. Neural Inf. Process. Syst.* (2017)

30. Wang, C., Liu, X., Yue, Y., Tang, X., Zhang, T., Jiayang, C., Yao, Y., Gao, W., Hu, X., Qi, Z., Wang, Y., Yang, L., Wang, J., Xie, X., Zhang, Z., Zhang, Y.: Survey on factuality in large language models: Knowledge, retrieval and domain-specificity (2023). <https://arxiv.org/abs/2310.07521>
31. Xiao, J., Ding, L., Barry, J., Elkaref, M., De Mel, G., Han, J.: Orag: Ontology-guided retrieval-augmented generation for theme-specific entity typing. In: First Conference on Language Modeling (2024)
32. Zhao, P., Zhang, H., Yu, Q., Wang, Z., Geng, Y., Fu, F., Yang, L., Zhang, W., Jiang, J., Cui, B.: Retrieval-augmented generation for ai-generated content: A survey. arXiv preprint arXiv:2402.19473 (2024)
33. Zhu, K., Luo, Y., Xu, D., Wang, R., Yu, S., Wang, S., Yan, Y., Liu, Z., Han, X., Liu, Z., et al.: Rageval: Scenario specific rag evaluation dataset generation framework. arXiv preprint arXiv:2408.01262 (2024)